

CURSO DE C++

Capítulo 1

Variables en C#

Desde principiante hasta nivel práctico

Enfoque: Desarrollo de C#

Capítulo técnico, educativo y profesional

Objetivo del capítulo

En este capítulo aprenderás qué son las variables en C#, por qué existen y cómo permiten que un programa conserve y manipule información mientras se ejecuta. El aprendizaje comienza con la idea de almacenar datos y avanza hacia declaración, inicialización, asignación, tipos de datos, inferencia con var, alcance, constantes y criterios profesionales para elegir nombres y tipos adecuados.

Al finalizar este capítulo serás capaz de:

- Explicar con tus propias palabras qué es una variable y qué problema resuelve.
- Declarar, inicializar y modificar variables utilizando la sintaxis correcta de C#.
- Elegir tipos básicos como int, double, decimal, bool, char y string según la información.
- Distinguir entre declaración, inicialización y asignación.
- Utilizar var entendiendo que el tipo sigue siendo determinado en compilación.
- Reconocer el alcance básico de una variable y evitar usar identificadores fuera de su contexto.
- Construir programas sencillos para sensores, inventarios, empleados y sistemas de producción.
- Detectar errores comunes y aplicar convenciones profesionales de nomenclatura.

1. Introducción al tema

¿Qué es una variable? Imagina una línea de producción. Durante el turno necesitas conservar la temperatura actual, la cantidad de piezas fabricadas y el estado de una máquina. Un programa necesita un mecanismo equivalente: una variable es un nombre que permite representar y acceder a un dato cuyo valor puede cambiar durante la ejecución.

¿Por qué existe? Sin variables, un programa tendría que repetir valores directamente en cada operación y no podría conservar de forma clara la información que obtiene, calcula o modifica. Las variables convierten los datos en elementos identificables.

Analogía: piensa en una variable como una caja con una etiqueta. La etiqueta es el nombre; el contenido es el valor; las reglas sobre qué puede guardarse están relacionadas con el tipo de dato. La analogía es útil, aunque internamente la representación depende del tipo y de la ejecución de .NET.

¿Dónde se usan? En prácticamente todo software: formularios, videojuegos, aplicaciones empresariales, cálculos científicos, automatización, telemetría, inventarios y sistemas industriales.

Relación con el curso: las variables serán la materia prima de operadores, decisiones, ciclos, métodos, arreglos y programación orientada a objetos. Antes de procesar información, normalmente debes poder representarla.

2. Concepto fundamental: tipo, nombre y valor

¿Qué es?

Una declaración básica combina un **tipo** y un **identificador** o nombre. El tipo comunica qué clase de dato representa la variable; el nombre permite referirse a ella.

¿Por qué se utiliza?

C# es un lenguaje con tipado estático: el compilador conoce y verifica el tipo de una variable. Esto ayuda a detectar incompatibilidades antes de ejecutar el programa.

¿Cuándo se utiliza?

Siempre que un valor deba conservarse, consultarse, calcularse o modificarse durante una parte del programa.

¿Cómo funciona?

Al compilar, C# analiza las declaraciones y comprueba que las operaciones sean compatibles con los tipos. Durante la ejecución, el programa administra los valores según las reglas del runtime. Como principiante, lo esencial es entender la relación: **nombre** → **variable** → **valor representado con un tipo**.

Sintaxis o estructura

```
tipo nombre;  
tipo nombre = valor;  
nombre = nuevoValor;
```

Ejemplo básico — Nivel 1

```
int piezasProducidas = 120;  
Console.WriteLine(piezasProducidas);
```

Línea 1: *int* indica un número entero. *piezasProducidas* es el nombre descriptivo. *= 120* inicializa la variable con 120.
Línea 2: se lee el valor actual y se envía a la consola.
Resultado esperado: 120.

3. Declaración, inicialización y asignación

Concepto	Qué ocurre	Ejemplo
Declaración	Se crea la variable con un tipo y nombre.	<code>int temperatura;</code>
Inicialización	Se proporciona su primer valor.	<code>int temperatura = 25;</code>
Asignación	Se establece o reemplaza el valor.	<code>temperatura = 30;</code>

```
int temperatura;  
temperatura = 25;  
temperatura = 30;
```

Después de la primera línea existe una variable llamada *temperatura*. Después de la segunda contiene 25. La tercera no crea otra variable: modifica el valor existente.

Error frecuente

```
int cantidad = 10;  
int cantidad = 20; // Error: el mismo nombre ya fue declarado en este ámbito
```

4. Tipos de datos fundamentales

Elegir un tipo es elegir cómo el programa interpretará y validará una categoría de información.

Tipo	Representa	Ejemplo	Uso típico
int	Enteros	150	Piezas, edades, contadores
long	Enteros de mayor rango	90000000000L	Contadores muy grandes
double	Decimales de precisión general	25.7	Mediciones y cálculos científicos
decimal	Decimales orientados a alta precisión decimal	125.50m	Importes monetarios

Tipo	Representa	Ejemplo	Uso típico
bool	Verdadero o falso	true	Estados y decisiones
char	Un carácter	'A'	Códigos de un carácter
string	Texto	"Motor"	Nombres y mensajes

Nota: el tipo correcto depende del significado y del rango necesario. Para dinero suele preferirse *decimal* porque trabaja con un modelo decimal adecuado para muchos cálculos financieros; no se debe elegir un tipo únicamente porque "parece funcionar".

Nivel 2 — Ejemplo intermedio

```
string nombreMaquina = "Prensa-01";
int piezasProducidas = 850;
double temperaturaActual = 67.5;
bool maquinaActiva = true;

Console.WriteLine(nombreMaquina);
Console.WriteLine(piezasProducidas);
Console.WriteLine(temperaturaActual);
Console.WriteLine(maquinaActiva);
```

Cada variable representa una clase diferente de información. El nombre describe el propósito y el tipo describe la naturaleza del valor. Al ejecutar, la consola mostrará los cuatro valores en líneas separadas.

5. Reasignación y evolución del estado

¿Qué es?

Reasignar significa cambiar el valor almacenado en una variable ya existente.

¿Por qué se utiliza?

Muchos sistemas representan información que cambia: velocidad de un motor, stock disponible o puntuación de un jugador.

```
int inventario = 100;
inventario = inventario - 5;
Console.WriteLine(inventario);
```

La expresión del lado derecho se evalúa primero usando el valor actual: $100 - 5 = 95$. Después el resultado se asigna nuevamente a *inventario*. Resultado esperado: 95.

Representación:

inventario = 100 → calcular $100 - 5$ → inventario = 95

Nivel 3 — Ejemplo práctico: estación de producción

```
string codigoEstacion = "EST-07";
int piezasIniciales = 500;
int piezasFabricadas = 125;
int piezasDisponibles = piezasIniciales + piezasFabricadas;
double temperatura = 68.4;
bool requiereRevision = false;

Console.WriteLine("Estación: " + codigoEstacion);
Console.WriteLine("Piezas disponibles: " + piezasDisponibles);
Console.WriteLine("Temperatura: " + temperatura);
Console.WriteLine("¿Requiere revisión?: " + requiereRevision);
```

El programa conserva datos de diferentes categorías y construye un valor derivado: *piezasDisponibles* depende de dos variables anteriores. Esto ilustra una idea central: las variables permiten que la información circule entre operaciones.

Posibles errores: usar *string* para una cantidad que debe calcularse; utilizar nombres ambiguos como *x1*; o cambiar un valor sin comprender que las expresiones se evalúan antes de la asignación.

6. Inferencia de tipo con var

¿Qué es?

var permite que el compilador infiera el tipo a partir de la expresión inicial.

```
var nombre = "Robot-12"; // string
var cantidad = 40; // int
var temperatura = 22.5; // double
```

Importante: *var* no significa que la variable pueda cambiar libremente de tipo.

```
var cantidad = 40;
// cantidad = "cuarenta"; // Error: cantidad fue inferida como int
```

¿Cuándo utilizarlo?

Puede mejorar la legibilidad cuando el tipo es evidente o excesivamente largo. Si oculta información importante para entender el código, un tipo explícito puede ser mejor.

No puedes escribir *var dato*; sin inicializador porque el compilador necesita una expresión para inferir el tipo.

7. Constantes: valores que no deben cambiar

Una **constante** representa un valor conocido que no debe modificarse después de declararse.

```
const int MaximoSensores = 10;
```

A diferencia de una variable ordinaria, no puedes reasignar una constante. Es útil para límites y valores fijos definidos como parte de una regla del programa.

```
// MaximoSensores = 20; // Error
```

Variable	Constante
Puede cambiar mediante asignación.	No puede reasignarse después de declararse.
Representa estado o información variable.	Representa un valor fijo conocido.
Ej.: temperaturaActual	Ej.: MaximoSensores

8. Alcance básico o scope

El **alcance** define dónde puede utilizarse un nombre. Una variable declarada dentro de un bloque delimitado por llaves normalmente solo puede usarse dentro de ese bloque.

```
if (true)
{
    int piezas = 10;
    Console.WriteLine(piezas);
}

// Console.WriteLine(piezas); // Error: está fuera de su alcance
```

Esta regla evita que una variable sea accesible desde cualquier parte del programa sin control y reduce confusiones sobre su ciclo de vida.

Nivel 4 — Ejemplo profesional: registro simplificado de producción

```
string lineaProduccion = "Línea A";
int metaDiaria = 1000;
int produccionActual = 820;
int piezasPendientes = metaDiaria - produccionActual;
decimal costoPorPieza = 12.50m;
decimal costoProduccion = produccionActual * costoPorPieza;
bool metaAlcanzada = produccionActual >= metaDiaria;

Console.WriteLine("=== REPORTE DE PRODUCCIÓN ===");
Console.WriteLine("Línea: " + lineaProduccion);
Console.WriteLine("Meta: " + metaDiaria);
Console.WriteLine("Producción actual: " + produccionActual);
Console.WriteLine("Pendientes: " + piezasPendientes);
Console.WriteLine("Costo estimado: " + costoProduccion);
Console.WriteLine("¿Meta alcanzada?: " + metaAlcanzada);
```

Este ejemplo se aproxima a un módulo de una aplicación mayor: datos de entrada, valores derivados y un estado booleano. En un sistema profesional estos valores podrían llegar desde una base de datos, sensores o una interfaz; aquí se declaran directamente para concentrarnos en las variables.

9. Diagramas y representación visual

Flujo general:

Dato inicial → Declaración/Inicialización → Uso en una expresión → Resultado → Nueva asignación opcional

Ejemplo: *produccionActual* = 820 → comparar con *metaDiaria* = 1000 → resultado *false* → guardar en *metaAlcanzada*.

La variable no es una instrucción aislada: suele formar parte de un flujo de datos.

Errores de confusión comunes

Se confunde...	Con...	Diferencia clave
Declarar	Asignar	Declarar introduce el nombre; asignar establece un valor.
var	Tipo dinámico	var sigue teniendo un tipo inferido y estático.
string	char	string representa texto; char representa un carácter entre comillas simples.
double	decimal	Ambos manejan fracciones, pero tienen representaciones y usos distintos; decimal suele elegirse para importes.
Variable	Constante	La variable puede cambiar; const no.

Errores comunes al trabajar con Variables

Usar una variable antes de inicializarla

Ocurre cuando se intenta leer un valor que el compilador no considera asignado. Corrige inicializando antes de usar y evita rutas de código ambiguas.

Elegir el tipo incorrecto

Guardar cantidades calculables como texto obliga a conversiones y puede producir errores. Identifica primero qué representa el dato.

Nombres poco descriptivos

Nombres como `a`, `dato1` o `x` pueden dificultar el mantenimiento. Usa nombres que expliquen la intención.

Confundir = con una comparación

El símbolo `=` asigna. Las comparaciones se estudian con operadores como `==`. Comprender la diferencia evita errores lógicos.

Reutilizar una variable para propósitos distintos

Una variable llamada `temperatura` no debería terminar almacenando un nombre de cliente. Mantener una responsabilidad clara mejora la comprensión.

Buenas prácticas

- Usa nombres descriptivos: `temperaturaActual` comunica más que `t`.
- En C# utiliza normalmente **camelCase** para variables locales y parámetros.
- Inicializa una variable cerca del lugar donde empieza a ser necesaria, evitando ámbitos innecesariamente grandes.
- Selecciona el tipo por significado, rango y operaciones previstas, no solo por comodidad.
- No abuses de `var`: úsalo cuando el tipo sea evidente y conserve la claridad.
- Evita valores mágicos repetidos; cuando un valor es una regla fija, considera una constante con nombre.
- Mantén una variable con un propósito claro para facilitar pruebas, mantenimiento y futuras modificaciones.
- Revisa conversiones entre tipos para evitar pérdida de información o errores de precisión.

Conectando conocimientos

Las variables se conectan con los fundamentos previos: la sintaxis permite escribir declaraciones correctas; las salidas muestran valores; los comentarios documentan decisiones; los tipos determinan qué clase de información se maneja. Después, los operadores transformarán estos valores, las decisiones evaluarán condiciones booleanas, los ciclos modificarán contadores y las clases agruparán variables como parte del estado de un objeto.

Ejercicios prácticos

Ejercicio 1 — Comprensión básica

Objetivo: declarar e inicializar variables.

Enunciado: crea variables para representar el nombre de una aplicación, su versión y el número de usuarios activos.

Requisitos: elige tipos adecuados y muestra los valores.

Pistas: identifica qué datos son texto y cuáles son cantidades enteras.

Ejercicio 2 — Aplicación

Objetivo: actualizar información almacenada.

Contexto: un almacén comienza con una cantidad de piezas y recibe un nuevo lote.

Requisitos: declara las variables necesarias, calcula el inventario final y muestra el resultado.

Restricción: no escribas directamente el resultado final.

Pista: utiliza una expresión con variables.

Ejercicio 3 — Integración

Representa un sensor industrial mediante variables para código, temperatura, presión y estado. Elige cada tipo y explica en comentarios por qué lo seleccionaste.

Ejercicio 4 — Caso práctico

Diseña la parte inicial de un sistema C# para una máquina de producción. Debe almacenar identificador, modelo, velocidad actual, piezas producidas, costo por pieza y estado de funcionamiento. Muestra un reporte. No necesitas clases todavía: concéntrate en representar correctamente la información.

Ejercicio 5 — Reto

Problema: crea un monitor de turno de producción.

Requisitos: representar turno, operador, meta, producción actual, piezas rechazadas, costo por pieza y estado de meta.

Reglas: los cálculos deben depender de variables; el código debe usar nombres descriptivos; utiliza al menos un valor decimal y uno booleano.

Funcionalidad esperada: mostrar un resumen coherente.

Pistas: primero lista los datos, después decide sus tipos y finalmente identifica valores derivados.

Mini proyecto del capítulo: Monitor básico de una máquina

Contexto

Una aplicación interna necesita mostrar un resumen simple de una máquina conectada a una línea de producción.

Objetivo

Aplicar variables para representar datos, calcular valores derivados y comunicar el estado actual.

Requisitos funcionales

- Código y nombre de máquina.
- Temperatura actual.
- Velocidad.
- Piezas producidas.
- Meta del turno.
- Costo por pieza.
- Estado activo o detenido.
- Cálculo de piezas pendientes y costo estimado.

Requisitos técnicos

- Elegir tipos adecuados.
- Usar nombres descriptivos.
- Usar una constante si identificas una regla fija.
- Mostrar un reporte organizado.

Paso 1

Declara únicamente las variables de identidad y estado. Antes de programar, escribe en papel qué representa cada dato y qué tipo necesita.

Paso 2

Agrega las variables numéricas de producción y realiza al menos un cálculo utilizando otras variables.

Paso 3

Incorpora el estado booleano y el valor monetario. Revisa si tus tipos representan correctamente la información.

Paso final

Construye el reporte completo y prueba modificando los valores iniciales. El programa debe actualizar los resultados derivados sin que tengas que editar manualmente cada salida.

Comprueba tus conocimientos

1. ¿Qué tres ideas básicas suelen participar en una declaración de variable?
2. Explica la diferencia entre declaración, inicialización y asignación.
3. ¿Qué tipo elegirías para una cantidad de piezas y por qué?
4. ¿Qué tipo representa verdadero o falso?
5. Analiza: `int cantidad = 10; cantidad = cantidad + 5;`. ¿Cuál es el valor final?
6. ¿Por qué `var nombre = "Ana"` no crea una variable sin tipo?
7. ¿Qué error conceptual existe en: `char letra = "A";`?
8. ¿Cuándo podría ser más adecuado decimal que double?
9. ¿Por qué una constante no debe usarse para información que cambia?
10. ¿Qué significa que una variable esté fuera de alcance?
11. Propón un nombre profesional para una variable que almacena la temperatura actual.
12. Analiza: `string piezas = "20";`. ¿Qué limitación aparece si deseas sumar cinco piezas directamente?
13. ¿Qué ocurre conceptualmente en: `stock = stock - vendidos;`?
14. ¿Por qué un nombre descriptivo ayuda al mantenimiento?
15. Diseña las variables necesarias para representar un dron: código, autonomía, peso máximo y estado de vuelo.

Respuestas y explicaciones

1. Tipo, nombre y, opcionalmente, un valor inicial.
2. Declarar introduce la variable; inicializar establece su primer valor; asignar establece o reemplaza un valor.
3. Normalmente int, porque una cantidad de piezas es un número entero dentro de un rango apropiado.

4. bool.
5. 15, porque primero se calcula $10 + 5$ y después el resultado se asigna.
6. El compilador infiere string a partir de la expresión "Ana".
7. char usa comillas simples y representa un carácter: 'A'.
8. Para importes monetarios y situaciones donde el modelo decimal sea apropiado.
9. Porque una constante expresa precisamente un valor que no debe reasignarse.
10. Que su nombre no puede utilizarse desde esa parte del código debido a las reglas del bloque o contexto.
11. temperaturaActual es una opción clara y consistente con camelCase.
12. Es texto; para cálculos numéricos necesitarías una representación numérica o una conversión.
13. Se toma el valor actual de stock, se resta vendidos y el resultado reemplaza el valor anterior.
14. Porque comunica intención y reduce el tiempo necesario para comprender y modificar el programa.
15. Una solución posible usa string para código, double para autonomía y peso si requieren fracciones, y bool para un estado de vuelo simple; la elección exacta depende de los requisitos.

Resumen del capítulo

Concepto	Para qué sirve	Cuándo utilizarlo	Error común
Variable	Representar información	Cuando un dato debe conservarse o procesarse	Elegir un tipo incorrecto
Tipo de dato	Definir naturaleza del valor	Al declarar información	Usar texto para cálculos
Inicialización	Dar un primer valor	Antes de leer la variable cuando sea necesario	Usar antes de asignar
Asignación	Modificar un valor	Cuando cambia el estado	Confundir = con comparación
var	Inferir el tipo	Cuando el tipo sea evidente	Creer que elimina el tipo
const	Representar valores fijos	Reglas o límites constantes	Intentar reasignarla
Alcance	Controlar dónde existe un nombre	Al organizar bloques y métodos	Usar fuera del bloque

Ideas clave que debes recordar

- Una variable representa información mediante un nombre y un tipo.
- Declarar no es lo mismo que inicializar ni que reasignar.
- El tipo debe elegirse según el significado del dato y las operaciones necesarias.
- Una asignación reemplaza el valor anterior después de evaluar la expresión.
- var utiliza inferencia de tipo; no convierte la variable en dinámicamente tipada.
- bool representa estados lógicos; string representa texto y char un solo carácter.
- decimal suele ser apropiado para muchos cálculos monetarios.

- Las constantes representan valores que no deben cambiar.
- El alcance determina dónde puede utilizarse una variable.
- Los nombres descriptivos son una herramienta de calidad del software.

Mapa de aprendizaje

Conocimiento previo → sintaxis, salidas, comentarios y tipos de datos

↓

Nuevo concepto → variables: nombre, tipo y valor

↓

Aplicación básica → declarar, inicializar y mostrar

↓

Aplicación práctica → sensores, producción, inventarios y costos

↓

Integración en proyectos → operadores, decisiones, ciclos y métodos

↓

Uso profesional → estado de objetos, validación, reglas de negocio y sistemas completos

Dominar variables prepara el siguiente paso: aprender a transformar y combinar datos con operadores y expresiones. A partir de ahí, el programa podrá tomar decisiones y responder a cambios del estado representado por sus variables.